
React Native

Navegacao, Rotas e Multiplas Telas

Material de Apoio Inicial - Desenvolvimento Mobile

```
+-----+ +-----+ +-----+
|  HOME  | ----> | CADASTRO | ----> | DETALHES |
+-----+ +-----+ +-----+
```

Professor: Luciano Rocha

React Native - Expo Go - React Navigation - AsyncStorage

Apresentacao

Este material foi elaborado para alunos que estao aprendendo desenvolvimento mobile com React Native. A proposta e mostrar, passo a passo, como transformar um aplicativo de uma unica tela em um aplicativo organizado com varias paginas.

A primeira parte apresenta os conceitos em um projeto simples. A segunda parte aplica esses conceitos no projeto da loja usado em sala. Ao final, o aluno devera entender como criar telas, navegar entre elas, enviar dados para outra pagina e organizar melhor o codigo.

Objetivo do material

Ensinar navegacao e rotas de forma progressiva, com exemplos pequenos antes de alterar o projeto maior da loja.

Sumario

1. Parte 1 - Conceitos fundamentais
2. 1. O que sao paginas em aplicativos
3. 2. O que e navegacao
4. 3. React Navigation
5. 4. Projeto simples com Home e Sobre
6. 5. Envio de dados entre telas
7. Parte 2 - Aplicando no projeto da loja
8. 6. Antes e depois da estrutura da loja
9. 7. Criando HomeScreen
10. 8. Criando CadastroScreen
11. 9. Criando DetalhesScreen
12. 10. Integracao com AsyncStorage
13. 11. Passo a passo final
14. 12. Exercicios de fixacao

PARTE 1

Conceitos fundamentais de paginas, rotas e navegacao

2. O que e navegacao?

Navegacao e o processo de sair de uma tela e abrir outra. Em sites, isso normalmente acontece com links. Em aplicativos mobile, usamos bibliotecas de navegacao.

No React Web, muitos projetos usam React Router. No React Native, a biblioteca mais comum para esse objetivo e o React Navigation.

```
React Web
|
+-- React Router

React Native
|
+-- React Navigation
```

Exemplo conceitual

Imagine uma loja mobile. O usuario abre a tela inicial, toca em um produto e depois visualiza os detalhes desse produto.

```
Home
|
| toca em um produto
v
Detalhes
|
| botao Voltar
v
Home
```

Importante

Rotas nao sao apenas botoes. Rotas definem quais telas existem e como o usuario pode se deslocar entre elas.

3. React Navigation

React Navigation e a biblioteca usada para trabalhar com multiplas telas no React Native. Ela permite criar uma estrutura de navegacao semelhante a uma pilha.

Dependencias usadas

```
{
  "dependencies": {
    "@react-navigation/native": "latest",
    "@react-navigation/native-stack": "latest",
    "react-native-screens": "latest",
    "react-native-safe-area-context": "latest"
  }
}
```

Cada dependencia tem uma funcao. A biblioteca principal e @react-navigation/native. O pacote native-stack cria a navegacao em pilha. Os pacotes react-native-screens e safe-area-context ajudam no funcionamento correto em dispositivos moveis.

Ideia de pilha

```
Pilha de telas

[ Detalhes ] <- tela atual
[ Cadastro ]
[ Home      ] <- primeira tela
```

Quando navegamos para uma nova tela, ela entra no topo da pilha. Quando usamos voltar, removemos a tela atual e retornamos para a anterior.

4. Projeto simples - antes e depois

Antes de mexer no projeto da loja, vamos entender o menor exemplo possivel.

Antes: uma unica tela

```
import { Text, View } from "react-native";

export default function App() {
  return (
    <View>
      <Text>Minha primeira tela</Text>
    </View>
  );
}
```

Nesse exemplo, o aplicativo mostra apenas uma tela. Nao existe navegacao.

Depois: preparando rotas

```
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

O que mudou?

O App.js deixou de ser apenas a tela principal e passou a ser o organizador das rotas do aplicativo.

5. Criando HomeScreen

Para organizar as telas, criamos uma pasta chamada screens. Dentro dela, cada arquivo representa uma pagina.

Estrutura

```
screens/  
  HomeScreen.js  
  SobreScreen.js  
App.js
```

Arquivo: screens/HomeScreen.js

```
import { View, Text, Button, StyleSheet } from "react-native";  
  
function HomeScreen({ navigation }) {  
  return (  
    <View style={styles.container}>  
      <Text style={styles.titulo}>Tela Inicial</Text>  
  
      <Button  
        title="Ir para Sobre"  
        onPress={() => navigation.navigate("Sobre")}  
      />  
    </View>  
  );  
}  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    padding: 20,  
    justifyContent: "center"  
  },  
  titulo: {  
    fontSize: 24,  
    fontWeight: "bold",  
    marginBottom: 20  
  }  
});  
  
export default HomeScreen;
```

O parametro navigation e recebido automaticamente pela tela quando ela esta registrada no Stack.Navigator.

6. Criando SobreScreen

Arquivo: screens/SobreScreen.js

```
import { View, Text, Button, StyleSheet } from "react-native";

function SobreScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Tela Sobre</Text>

      <Text>
        Este aplicativo demonstra navegacao entre telas.
      </Text>

      <Button
        title="Voltar"
        onPress={() => navigation.goBack()}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    justifyContent: "center"
  },
  titulo: {
    fontSize: 24,
    fontWeight: "bold",
    marginBottom: 20
  }
});

export default SobreScreen;
```

Registrando as duas telas

```
import HomeScreen from "../screens/HomeScreen";
import SobreScreen from "../screens/SobreScreen";

<Stack.Navigator>
  <Stack.Screen name="Home" component={HomeScreen} />
  <Stack.Screen name="Sobre" component={SobreScreen} />
</Stack.Navigator>
```

Erro comum

O nome usado em `navigation.navigate('Sobre')` precisa ser exatamente igual ao `name='Sobre'` registrado no `Stack.Screen`.

7. Enviando dados para outra tela

Muitas vezes precisamos abrir uma tela e enviar informacoes para ela. Por exemplo: tocar em um produto e abrir a tela de detalhes desse produto.

Enviando dados

```
navigation.navigate("Detalhes", {  
  nome: "Tenis Esportivo",  
  preco: "199.90"  
});
```

Recebendo dados

```
function DetalhesScreen({ route }) {  
  const { nome, preco } = route.params;  
  
  return (  
    <View>  
      <Text>{nome}</Text>  
      <Text>R$ {preco}</Text>  
    </View>  
  );  
}
```

```
HomeScreen  
  |  
  | navigation.navigate("Detalhes", { produto })  
  v  
DetalhesScreen  
  |  
  | const { produto } = route.params  
  v  
Mostra dados do produto
```

PARTE 2

Aplicando navegacao no projeto atual da loja

8. Projeto da loja - estrutura antiga

Antes da navegacao, o projeto da loja concentrava quase tudo no App.js. O App tinha a lista de produtos, formulario, botoes e exibicao dos cards.

```
App.js
|
+-- Navbar
+-- Formulario
+-- ProductList
|   +-- ProductCard
+-- Produto selecionado
```

Problema

Quando o App.js faz tudo, o codigo fica grande demais. Isso dificulta leitura, correcao de erros e manutencao.

Objetivo da refatoracao

Vamos transformar o projeto em tres telas principais: HomeScreen, CadastroScreen e DetalhesScreen.

```
App.js
|
+-- HomeScreen
+-- CadastroScreen
+-- DetalhesScreen
```

9. Nova estrutura de pastas

Antes

```
App.js
components/
  Navbar.js
  ProductList.js
  ProductCard.js
```

Depois

```
App.js
components/
  ProductList.js
  ProductCard.js

screens/
  HomeScreen.js
  CadastroScreen.js
  DetalhesScreen.js
```

A pasta components continua sendo usada para partes reutilizaveis da interface. A pasta screens passa a guardar as paginas do aplicativo.

Dica

Componentes sao partes reaproveitaveis. Telas sao paginas completas do aplicativo.

10. Alterando o App.js

O App.js passa a ser o centro de controle das rotas e dos dados principais.

Remover do App.js

Remova do App.js o formulario completo e a lista renderizada diretamente. Esses elementos serao movidos para telas separadas.

Adicionar imports

```
import { useEffect, useState } from "react";
import AsyncStorage from "@react-native-async-storage/async-storage";

import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";

import HomeScreen from "../screens/HomeScreen";
import CadastroScreen from "../screens/CadastroScreen";
import DetalhesScreen from "../screens/DetalhesScreen";
```

Criar Stack

```
const Stack = createNativeStackNavigator();
```

O Stack sera usado para registrar as telas do aplicativo.

11. App.js completo com rotas

```
const STORAGE_KEY = "produtos";

export default function App() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    carregarProdutos();
  }, []);

  async function carregarProdutos() {
    const dadosSalvos = await AsyncStorage.getItem(STORAGE_KEY);

    if (dadosSalvos) {
      setProducts(JSON.parse(dadosSalvos));
    }
  }

  async function salvarProdutos(lista) {
    await AsyncStorage.setItem(
      STORAGE_KEY,
      JSON.stringify(lista)
    );
  }

  function atualizarLista(novaLista) {
    setProducts(novaLista);
    salvarProdutos(novaLista);
  }

  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" options={{ title: "Minha Loja" }}>
          {(props) => (
            <HomeScreen {...props} products={products} />
          )}
        </Stack.Screen>

        <Stack.Screen name="Cadastro" options={{ title: "Cadastro" }}>
          {(props) => (
            <CadastroScreen
              {...props}
              products={products}
              atualizarLista={atualizarLista}
            />
          )}
        </Stack.Screen>

        <Stack.Screen
          name="Detalhes"
          component={DetalhesScreen}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Importante

A lista products fica no App.js porque precisa ser compartilhada entre as telas.

12. Criando HomeScreen

A HomeScreen sera a tela inicial. Ela lista os produtos e permite acessar a tela de cadastro.

```
import { View, Text, Button, StyleSheet } from "react-native";
import ProductList from "../components/ProductList";

function HomeScreen({ navigation, products }) {
  function abrirDetalhes(produto) {
    navigation.navigate("Detalhes", {
      produto: produto
    });
  }

  return (
    <View style={styles.screen}>
      <Text style={styles.title}>Produtos</Text>

      <Text style={styles.subtitle}>
        Total de produtos: {products.length}
      </Text>

      <Button
        title="Novo Produto"
        onPress={() => navigation.navigate("Cadastro")}
      />

      <ProductList
        products={products}
        aoSelecionar={abrirDetalhes}
      />
    </View>
  );
}

export default HomeScreen;
```

O que acontece?

O botao Novo Produto abre a tela Cadastro. Ao tocar em um card, a funcao abrirDetalhes envia o produto para a tela Detalhes.

13. ProductList e ProductCard

A lista continua separada em componentes. Isso evita repetir codigo nas telas.

ProductList

```
function ProductList({ products, aoSelecionar }) {  
  return (  
    <FlatList  
      data={products}  
      keyExtractor={({product}) => product.id}  
      renderItem={({ item }) => (  
        <ProductCard  
          item={item}  
          aoSelecionar={aoSelecionar}  
        />  
      )}  
    />  
  );  
}
```

ProductCard

```
function ProductCard({ item, aoSelecionar }) {  
  return (  
    <TouchableOpacity  
      style={styles.card}  
      onPress={() => aoSelecionar(item)}  
    >  
      <Text>{item.name}</Text>  
      <Text>R$ {item.price}</Text>  
      <Text>Categoria: {item.category}</Text>  
    </TouchableOpacity>  
  );  
}
```

Fluxo

HomeScreen envia abrirDetalhes para ProductList. ProductList envia para ProductCard. ProductCard executa ao tocar no produto.

14. Criando CadastroScreen

A tela Cadastro sera responsavel por cadastrar, editar e excluir produtos.

Estados principais

```
const [nomeProduto, setNomeProduto] = useState("");
const [precoProduto, setPrecoProduto] = useState("");
const [categoriaProduto, setCategoriaProduto] = useState("");
const [quantidadeProduto, setQuantidadeProduto] = useState("");
const [produtoEditando, setProdutoEditando] = useState(null);
```

Formulario

```
<TextInput
  placeholder="Nome"
  value={nomeProduto}
  onChangeText={setNomeProduto}
/>

<TextInput
  placeholder="Preco"
  value={precoProduto}
  onChangeText={setPrecoProduto}
/>

<Button
  title={produtoEditando ? "Atualizar" : "Cadastrar"}
  onPress={salvarProduto}
/>
```

O texto do botao muda dependendo do estado produtoEditando.

15. Salvando, editando e excluindo

Cadastrar novo produto

```
const novoProduto = {
  id: Date.now().toString(),
  name: nomeProduto,
  price: precoProduto,
  category: categoriaProduto,
  quantity: quantidadeProduto,
  image: "https://placeholder.co/300x220/222222/ffffff/png?text=Produto"
};

atualizarLista([...products, novoProduto]);
```

Editar produto

```
const listaAtualizada = products.map((produto) => {
  if (produto.id === produtoEditando.id) {
    return {
      ...produto,
      name: nomeProduto,
      price: precoProduto,
      category: categoriaProduto,
      quantity: quantidadeProduto
    };
  }
  return produto;
});

atualizarLista(listaAtualizada);
```

Excluir produto

```
const novaLista = products.filter(
  (produto) => produto.id !== id
);

atualizarLista(novaLista);
```

16. Criando DetalhesScreen

A tela de detalhes recebe um produto pela navegacao e mostra as informacoes completas.

```
import { View, Text, Image, Button } from "react-native";

function DetalhesScreen({ route, navigation }) {
  const { produto } = route.params;

  return (
    <View>
      <Image source={{ uri: produto.image }} />

      <Text>{produto.name}</Text>
      <Text>R$ {produto.price}</Text>
      <Text>Categoria: {produto.category}</Text>
      <Text>Estoque: {produto.quantity}</Text>

      <Button
        title="Voltar"
        onPress={() => navigation.goBack()}
      />
    </View>
  );
}

export default DetalhesScreen;
```

Erro comum

Se route.params estiver vazio, provavelmente a tela foi aberta sem enviar o produto em navigation.navigate().

17. AsyncStorage com multiplas telas

Mesmo com varias telas, os dados continuam salvos no App.js e persistidos no AsyncStorage.

```
CadastroScreen
  |
  | atualizarLista(novaLista)
  v
App.js
  |
  | setProducts(novaLista)
  | salvarProdutos(novaLista)
  v
AsyncStorage
```

A funcao atualizarLista e enviada para CadastroScreen por props. Assim, a tela de cadastro consegue alterar a lista principal.

Importante

Quando uma informacao precisa ser usada por varias telas, ela deve ficar em um local acima dessas telas, como o App.js.

18. Passo a passo final

Use esta sequencia quando for adaptar um projeto de uma tela para um projeto com rotas.

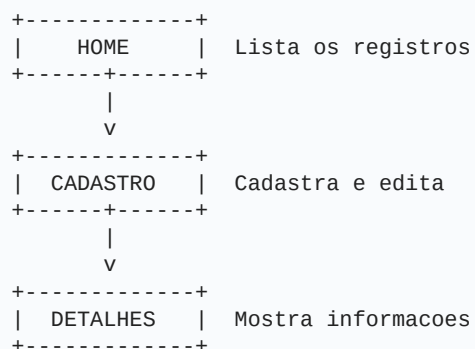
1. Instalar dependencias de navegacao.
2. Criar a pasta screens.
3. Criar HomeScreen.js.
4. Criar CadastroScreen.js.
5. Criar DetalhesScreen.js.
6. Importar NavigationContainer.
7. Criar Stack com createNativeStackNavigator.
8. Registrar as telas no Stack.Navigator.
9. Usar navigation.navigate() para trocar de tela.
10. Usar route.params para receber dados.
11. Testar Home -> Cadastro.
12. Testar Home -> Detalhes.
13. Testar Voltar.
14. Testar cadastro e persistencia.

19. Exercicios de fixacao

1. Explique com suas palavras o que e uma rota.
2. Qual a funcao do NavigationContainer?
3. Para que serve o Stack.Navigator?
4. O que acontece quando usamos navigation.navigate('Cadastro')?
5. Como enviar um produto para a tela Detalhes?
6. Como receber dados usando route.params?
7. Qual a funcao do navigation.goBack()?
8. Por que criamos a pasta screens?
9. Qual a diferenca entre componente e tela?
10. Por que a lista products ficou no App.js?

20. Desafio final

Adapte o projeto da sua equipe para usar tres telas principais:



Temas possiveis: biblioteca, cadastro de alunos, controle financeiro, pet shop, lista de tarefas, agenda ou controle de estoque.

Orientacao final

Um projeto simples, organizado e funcionando vale mais do que um projeto complexo incompleto.